Bloons Tower Defense

Université de Lille

Projet présenté par

SYLLA Mariama Séré / SOW Farmata Sall

BEYE Khadim / SOW Mamadou Baïllo

Plateaux • Tours • Ballons • Évolutions

Plan

Les points essentiels de la présentation

Université de Lille

1. État d'avancement

ClassicalBoard & RandomBoard, chemins calculés

2. Modélisation & Architecture

Hiérarchie Board / Tower / Balloon

3. Extensibilité du code

Ajouter une tour sans toucher au code existant

4. Fonctionnement en équipe

Git, branches, réunions et organisation

5. Difficultés rencontrées

Bugs résolus et solutions appliquées

6. Bilan et perspectives

Ce qu'on a appris, nos fiertés

État d'avancement

Tout ce qui est réalisé dans ce livrable

Université de Lille

✅



Plateaux

- ClassicalBoard : chemin fixe calculé aléatoirement
- RandomBoard : chemins horizontaux/verticaux multiples

✅



Ballons

- Déplacement continu (progression fractionnaire)
- Vitesse et santé variables
- Trajectoire stockée individuellement

✅



Tours (7 types)

- DartMonkey, TackShooter, SniperMonkey
- BombTower, SuperMonkey, IceTower, SlowDownTower
- Effets : dégâts, gel, ralentissement

✅



Évolutions

- Augmentation portée / cadence / puissance
- Changement de projectile
- Application et suppression

✅



Actions joueur

- Achat et placement de tours
- Évolution / désévolution
- Vente d'une tour

Modélisation & Architecture

Hiérarchie des classes et principes de conception

Université de Lille

Hiérarchie des classes

Board (abstract)

- ClassicalBoard — chemin unique
- RandomBoard — chemins multiples

Tower (abstract)

- ProjectileTower
  - DartMonkeyTower
  - BombTower
  - SniperMonkeyTower
  - SuperMonkeyTower
  - TackShooterTower
- IceTower — effet gel
- SlowDownTower — effet ralentissement

Balloon

- Level (enum) — vitesse / santé
- Trajectoire — List<Position>
- Progression — indice + fraction

Principes clés

Héritage

Chaque type de tour hérite de Tower ou ProjectileTower. Les plateaux héritent de Board.

Polymorphisme

La méthode attackBallon() est redéfinie dans chaque tour, permettant des comportements différents à l'appel.

Copie défensive

La trajectoire est copiée à la création du ballon (new ArrayList<>(path)) pour éviter les mutations partagées.

Gestion du temps réel

timeLastAttack + canAttack() garantissent une cadence correcte sans surcharger la boucle de jeu.

Extensibilité du code

Ajouter une nouvelle tour sans modifier le code existant

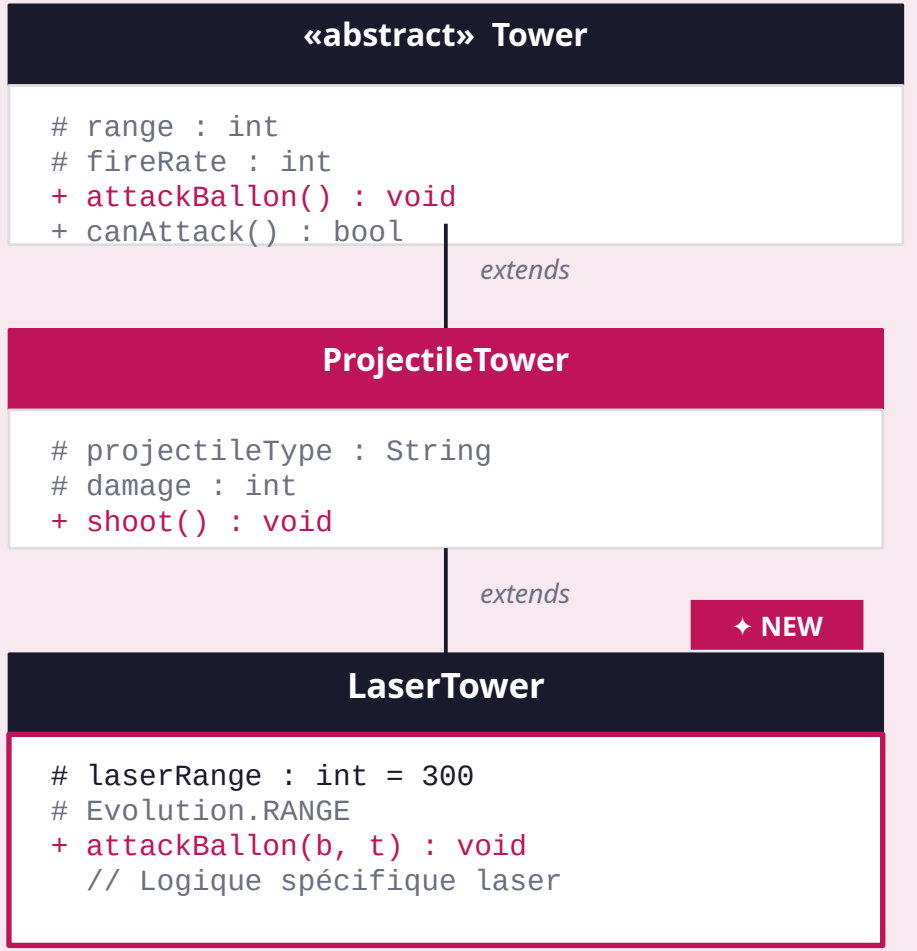
Université de Lille

Comment ajouter une nouvelle tour ?

1. Hériter de Tower (ou ProjectileTower)
2. Implémenter attackBallon()
3. Définir possibleEvolutions
4. ✅ Prêt à l'emploi !

Aucune modification du code existant nécessaire !

Diagramme UML · LaserTower



Difficultés rencontrées

Problèmes identifiés et solutions appliquées

Université de Lille

Catégories de problèmes

Bugs mémoire

Mutations inattendues des listes partagées.

Logique de jeu

Cadence des tours et synchronisation des effets.

Algorithme

Détection des chevauchements de chemins.

Git & équipe

Conflits de branches lors des merges.

Problème → Solution

⚠️ Bug positions mutées

→ Copie défensive : new ArrayList<>(path)

⚠️ Cadence trop rapide

→ timeLastAttack + canAttack()

⚠️ Chevauchement des chemins

→ Méthode hasOverLap() avec HashSet

⚠️ Restauration vitesse

→ Stockage vitesse originale + timer

⚠️ Conflits Git

→ Branches par fonctionnalité + rebase

Bilan & Perspectives

Ce qu'on retient de ce projet

Université de Lille

📌 Ce qu'on a appris

- Gestion de projet en équipe (Git, réunions)
- Conception orientée objet appliquée
- Gestion du temps réel dans un jeu
- Importance des tests unitaires

🎉 Nos fiertés

Architecture extensible

Ajouter une tour sans toucher au code existant

0 crash

Stabilité totale de la simulation

Code documenté

Javadoc complète sur toutes les classes

Merci pour votre attention • Questions ?